The National University of Computer and Emerging Sciences Peshawar Campus

Introduction to Structures

Object-Oriented Programming (CS-1004)

Muhammad Zulqarnain

Department of Computer Science

Lecture slide credit goes to Dr. Akhtar Jamil from the National University of Computer and Emerging Sciences, Islamabad (Email: akhtar.jamil@nu.edu.pk).

Motivation

- **Data Types**
- C++ has several *primitive data types*, or data types that are defined as a basic part of the language

bool	short int	long long int
char	unsigned short int	unsigned long long int
char_16_t	int	
char_32_t	unsigned int	float
wchar_t	long int	double
unsigned char	unsigned long int	long double

Motivation

- You have written programs that keep data in individual variables.
- Group items of same data type together, in C++ use arrays.
- How to store items of different data types together.
- For example, for employee following information may be needed.

Motivation

Variable Definition	Data Held
`int empNumber;`	Employee number
`string name;`	Employee's name
`double hours;`	Hours worked
`double payRate;`	Hourly pay rate
`double grossPay;`	Gross pay

Structure

- A **Structure** is a **container**, it can **hold a bunch of *things***.
  - These **things** can be of **any type**.

- **Structures** are used to **organize related data** (variables) into a **nice neat package**.

- Before a structure can be used, it must be declared.

Introducing Structures

- A **structure** is a collection and is referenced with **single name**.

- The data items in structure are called structure **members**, **elements**, or **fields**.

- The difference between **array** and **structure**: is that ***array consists*** *of a* ***set of values*** *of same data type* but on the other hand, **structure may consist of different data types**.

Introducing Structures

```
struct name
{
    variable declaration;
    // ... more declarations
    // may follow...
};
```

Introducing Structures

```
struct Employee
{
   int iEmpID; // Employee number
   string sName; // Employee's name
   double dWorkHours; // Hours worked
   double dPayRate; // Hourly pay rate
   float fSalary; // Gross pay
};
```

- This declaration declares a structure named Employee. The structure has five members.
- Employee is a new data type

Defining the Structure

- Structure definition tells how the structure is organized: it specifies what members the structure will contain.

Syntax & Example

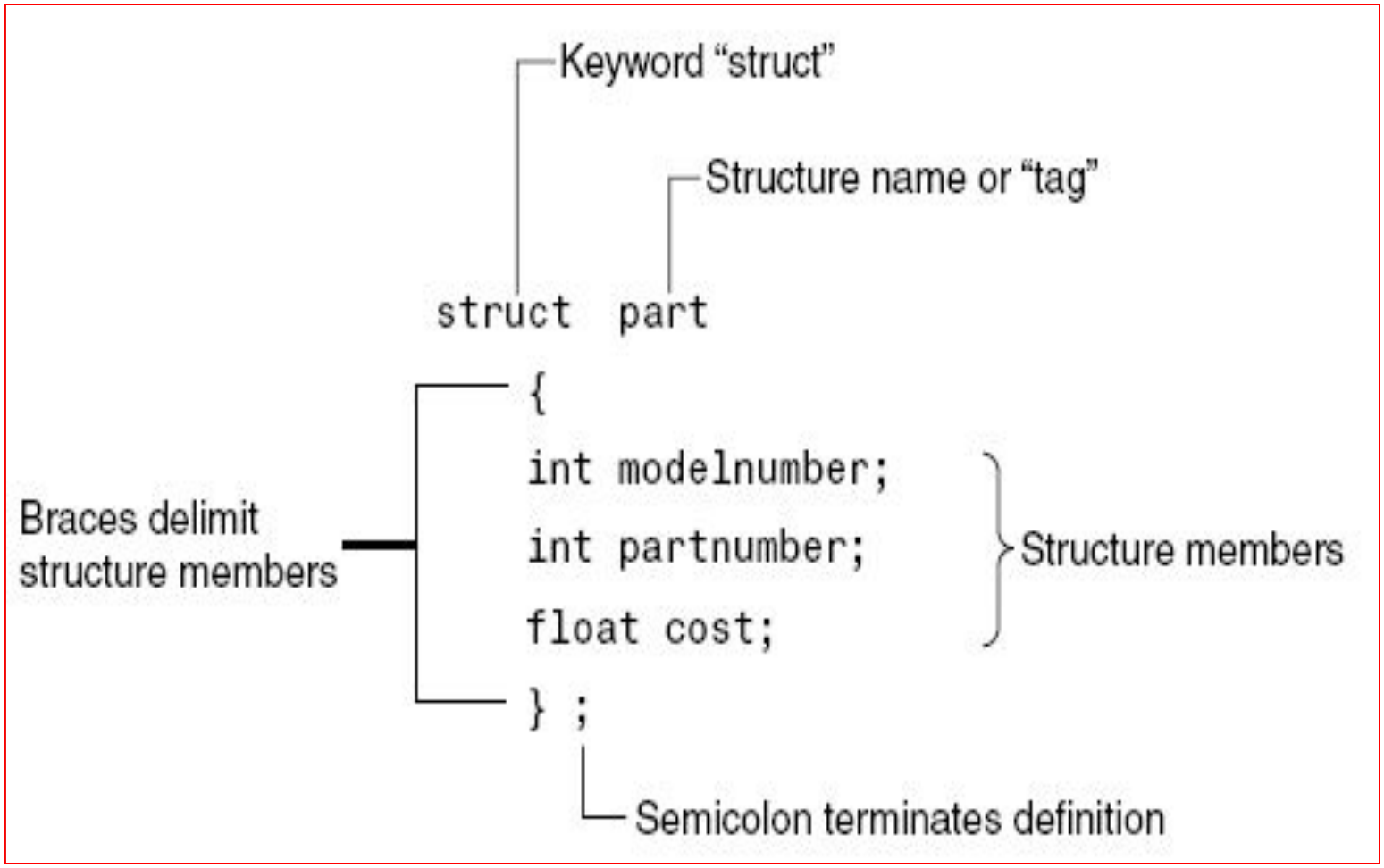
Syntax:

```
struct  StructName
{
    DataType1   Identifier1;
    DataType2   Identifier2;
    .
    .
    .
};
```

Example:

```
struct   Product
{
      int       ID;
      string   name;
      float     price;
};
```

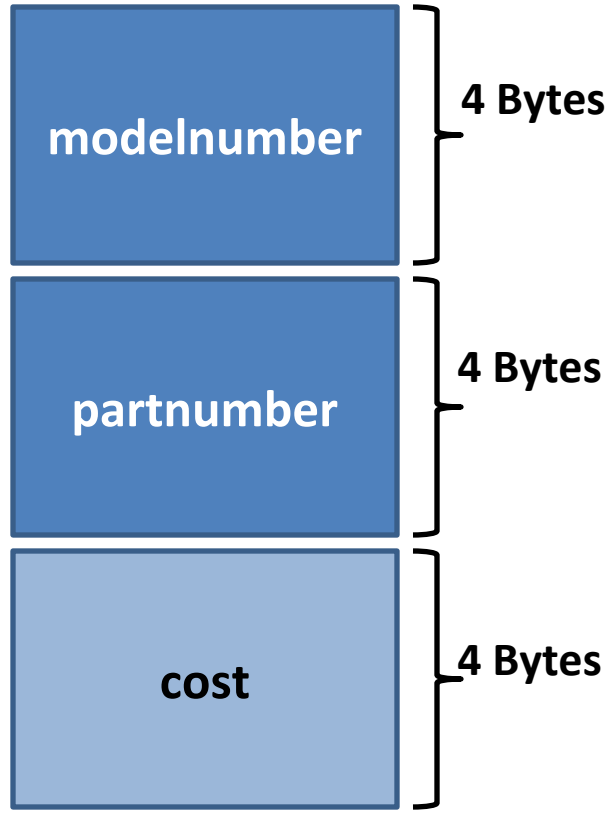
Structure Definition Syntax



Structure members in memory

Memory is only allocated when we create struct type variables

```
struct   part
{
    int       modelnumber;
    int       partnumber;
    float    cost;
};
```



Declaring a Structure variable

- **Structure variable** can be **created** after the **definition** of a **structure**. The **syntax** of the **declaration** is:

```
StructName    Identifier;
```

Example:

```
Employee emp; // create one structure variable
part    processor;
part    keyboard;
```

Another way of Declaring a Structure variable

- You can also **declare struct variables** during **definition** of the **struct**. For example:

```
struct   part
{
    int       modelnumber;
    int       partnumber;
    float   cost;
} part1;
```

- These statements define the struct named **part** and also declare **part1** to be a variable of type **part**.

Examples

```
struct Employee
{
    int iEmpID;
    string sName;
    double dWorkHours;
    double dPayRate;
    float fSalary;
} emp;
```

```
struct   Student
{
    string      firstName;
    string      lastName;
    char        courseGrade;
    int         Score;
    double      CGPA;

} std1, sts2;
```

```
struct Time
{
    int hour;
    int minutes;
    int seconds;
};

Time now;
```

```
struct  Date
{
    int day;
    int month;
    int year;
};

Date today;
```

```
struct Point
{
    int x,y;
};

Point p;
```

Initializing Structure Variables

- The syntax of **initializing structure** is:

```
StructName  struct_identifier  =  {Value1, Value2, …};
```

Structure Variable Initialization with Declaration

```
struct   Student
{
      string    firstName;
      string    lastName;
      char      courseGrade;
      int       marks;
};

void main( )
{
      Student s1= {“M”, “Umar”, ‘A’, 94} ;
}
```

Note: Values should be written in the same sequence in which they are specified in structure definition.

Structure Variable Initialization with Declaration

```
struct Employee
{
   int iEmpID; // Employee number
   string sName; // Employee's name
   double dWorkHours; // Hours worked
   double dPayRate; // Hourly pay rate
   float fSalary; // Gross pay
};

Employee emp = {101,"Ali",10,100.50,50000};
```

Assigning Values to Structure Variables

- After creating structure variable, values to structure members can be assigned using **dot (.) operator**

- The syntax is as follows:

```
student s1;

s1.firstName = "Muhammad";
s1.lastName = "Umar";
s1.courseGrade = 'A';
s1.marks = 93;
```

Assigning Values to Structure Variables

- After creating structure variable, values to structure members can be assigned using ***cin***

- Output to screen using ***cout***

```
student s1;

cin>>s1.firstName;
cin>>s1.lastName;
cin>>s1.courseGrade;
cin>>s1.marks ;

cout<<s1.firstName<<s1.lastName;
```

Assigning Values to Structure Variables

```
Employee emp;
emp.iEmpID = 1;
emp.sName = "Ali Khan";
emp.fSalary = 10000;
emp.dPayRate = 100;
emp.dWorkHours = 8;
```

Accessing Values of Structure Variables

- Dot operator (.) is used to access the values as well.

```
cout << "Name = " << emp.sName
     << " Salary = " << emp.sName;
```

Assigning one Structure Variable to another

- A **structure variable** can be **assigned** to **another structure variable** ***only if both are of same type***

- A **structure variable** can be **initialized** by **assigning** another **structure variable** to it by **using** the **assignment operator** as follows:

Example:

```
studentType   newStudent = {"Amir", "Ali", 'A', 98} ;
studentType   student2 = newStudent;
```

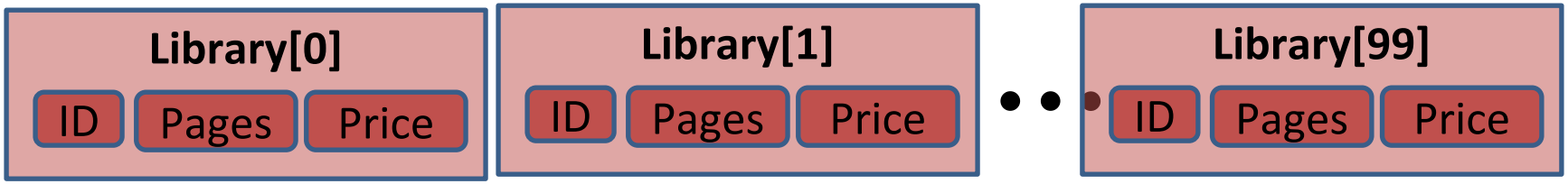
Assigning one Structure Variable to another

```
// structure
   Employee emp; // create one structure variable
   emp.iEmpID = 1;
   emp.sName = "Ali Khan";
   emp.fSalary = 10000;
   emp.dPayRate = 100;
   emp.dWorkHours = 8;


   Employee emp1 = emp;
   emp1.sName = "New Employee";
```

Array of Structures

An array of **structure** is a **type of array** in which **each element** contains a **complete structure**.

```
struct Book
{
    int    ID;
    int    Pages;
    float  Price;
};
Book   Library[100];    // declaration of array of structures
```



Initialization of Array of Structures

```
struct Book
{
    int    ID;
    int    Pages;
    float  Price;
};
Book   b[3];      // declaration of array of structures
```

Initializing can be at the time of declaration

```
Book   b[3] = {{1,275,70},{2,600,90},{3,786,100}};
```

- Or can be assigned values using ***cin***:

```
cin>>b[0].ID ;
    cin>>b[0].Pages;
    cin>>b[0].Price;
```

Partial Initialization of Array of Structures

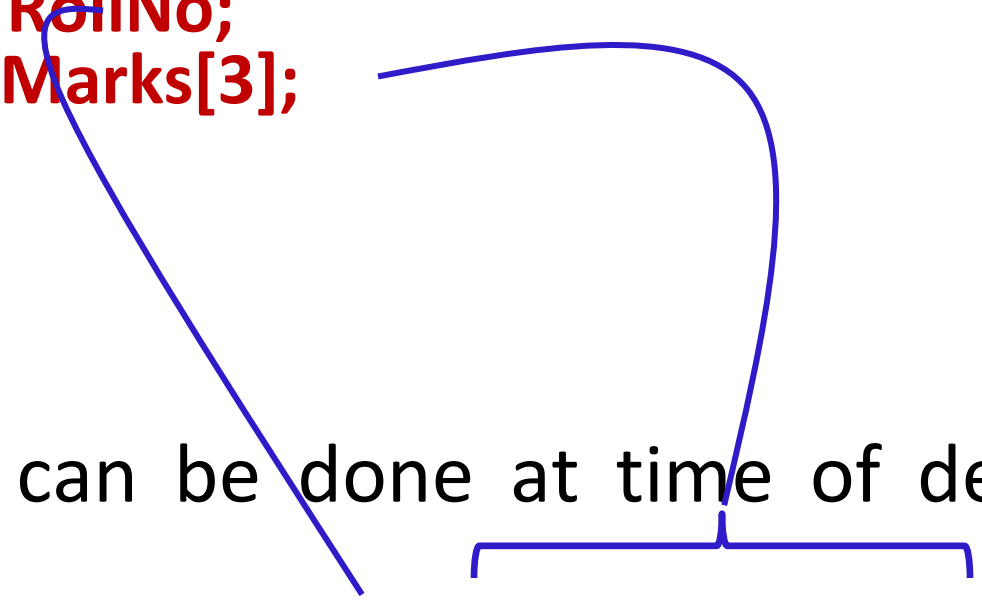
```
int main()
{
    struct  Book
    {
          int      ID;
          int      Pages;
          float  Price;
    };

    Book    b[4] = {{2}, {5,6,7},{}, {3,786,100}};
    for(int i=0;i<4;i++)
    {
        cout<<b[i].ID<<endl;
        cout<<b[i].Pages<<endl;
        cout<<b[i].Price<<endl;
        cout<<"---------------\n";
    }
    return 0;
}
```

```
2
0
0
---------------
5
6
7
---------------
0
0
0
---------------
3
786
100
---------------
```

Array as Member of Structures

- A **structure** may also **contain arrays** as members.

```
struct   Student
{
        int      RollNo;
        float    Marks[3];
 };
```



- Initialization can be done at time of declaration:

```
Student  S  = {1,  {70.0, 90.0, 97.0} };
```

Array as Member of Structures

- Or it can be **assigned values later** in the **program**:

```
Student   S;
S.RollNo = 1;
 S.Marks[0] = 70.0;
S.Marks[1] = 90.0;
 S.Marks[2] = 97.0;
```

- Or **user** can **use *cin*** to get **input directly**:

```
cin>>S.RollNo;
cin>>S.Marks[0];
cin>>S.Marks[1];
cin>>S.Marks[2];
```

Array as Member of Structures

```
Employee list[5];

  for (inti = 0; i < 5; i++)
  {
     list[i].iEmpID = i + 1;
     list[i].sName = "Employee " + to_string(i+1);
     list[i].fSalary = (i+1)*5000;
     list[i].dPayRate = 100;
     list[i].dWorkHours = 8;
  }
```

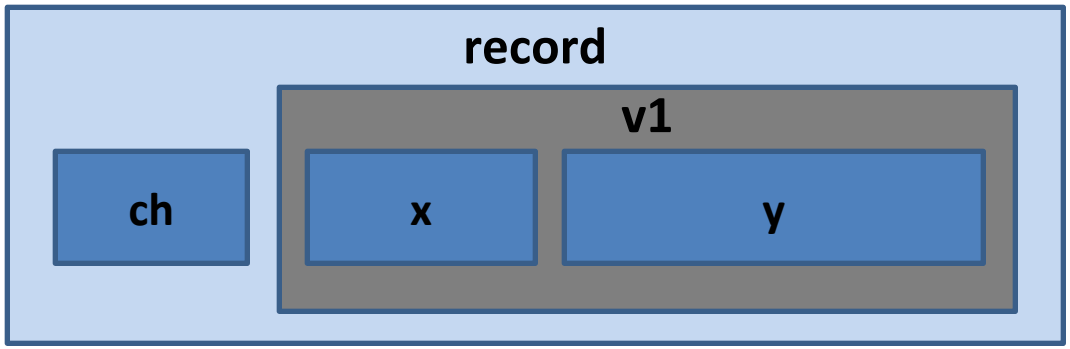
Array as Member of Structures

```
for (int i = 0; i < 5; i++)
  {
      cout << "ID = " << list[i].iEmpID
       << " Name = " << list[i].sName
       << "  Salary = " << list[i].fSalary
       << "  PayRate = " << list[i].dPayRate
       << "  WorkHours = " << list[i].dWorkHours
       <<endl;
  }
```

Nested Structure

- A structure can be a member of another structure: called nesting of structure.

```
struct   A
{
     int      x;
     double   y;
};

struct   B
{
      char    ch;
      A       v1;
};

B  record;
```



Initializing/Assigning to Nested Structure

```
struct   A{
        int     x;
        float  y;
};

struct B{
      char ch;
      A    v2;
};
```

```
void main() // Initialization
{
     B  record = {'S', {100, 3.6} };
}
```

```
void main() // Input
{
      B   record;
      cin>>record.ch;
      cin>>record.v2.x;
      cin>>record.v2.y;
}
```

```
void main() // Assignment
{
      B     record;
      record.ch = 'S';
      record.v2.x = 100;
      record.v2.y = 3.6;
}
```

Accessing Structures with Pointers

- **Pointer variables** can be **used** to **point** to **structure** type variables too.
- The pointer variable should be of same type, for example: structure type

```
struct   Rectangle {
        int    width;
        int    height;
};

void main( )
{
      Rectangle rect1={22,33};
      Rectangle* rect1Ptr = &rect1;
}
```

Accessing Structures with Pointers

- How to access the structure members (using pointer)?
  - Use **dereferencing operator** (*) with **dot** (.) operator

```
struct   Rectangle {
        int    width;
        int    height;
};

void main( )
{
      Rectangle rect1={22,33};
      Rectangle* rectPtr = &rect1;
      cout<< (*rectPrt).width<<(*rectPrt).height;
}
```

Accessing Structures with Pointers

- **Is there some easier way also?**
  - Use **arrow operator** (**->**)

```
struct   Rectangle {
        int    width;
        int    height;
};

void main( )
{
      Rectangle rect1={22,33};
      Rectangle* rectPtr = &rect1;
      cout<< rectPrt->width<<rectPrt->height;
}
```

Accessing Structures with Pointers

```
Point p;
p.x = 10;
p.y = 20;

   Point* ptr = &p;
ptr->x = 10;
ptr->y = 20;
```

```
cout << "Point (x,y) = (" << p.x << "," << p.y << ")" <<endl;
cout << "Point (x,y) = (" << (*ptr).x << "," << (*ptr).y << ")" << endl;
cout << "Point (x,y) = (" << ptr->x << "," << ptr->y << ")" << endl;
```

Output

```
Point (x,y) = (10,20)
Point (x,y) = (10,20)
Point (x,y) = (10,20)
```

Comparing Structure Variables

- You cannot perform comparison operations directly on structure variables.

```
Point p1, p2;
   if (p1==p2) // Error
   {
      cout<<"Yes"
   }
if (p1.y = p2.y && p1.x == p2.x) // no issues
   {
      cout << "Yes";
   }
```

Anonymous Structure

- **Structures** can be **anonymous**:

```
struct
{
   int x;
   int y;
} p1,p2;

p1.x=10;
p1.y=20;
p2=p1;
cout<<"\nX in p2="<<p2.x<<" and Y in p2="<<p2.y;
```

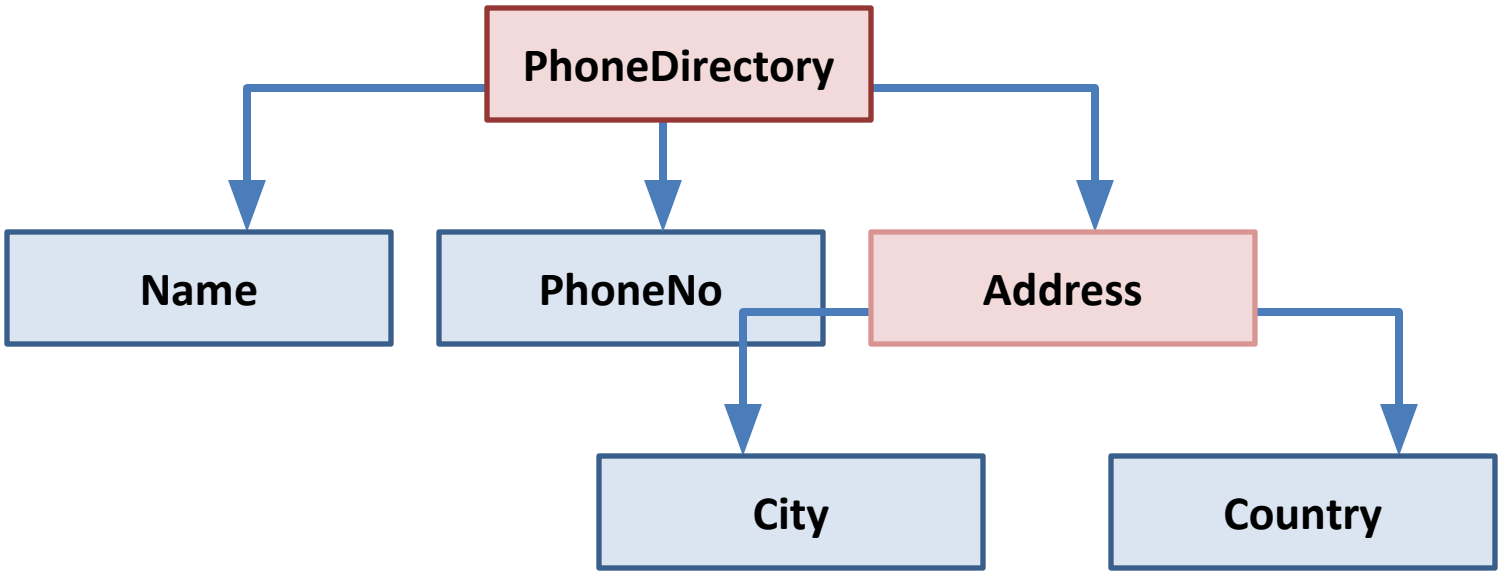
Practice Question 1

- Define a structure called **“car”**. The member elements of the car structure are:
  - string Model;
  - int Year;
  - float Price

Create an array of 30 cars. Get input for all 30 cars from the user. Then the program should display complete information (***Model***, ***Year***, ***Price***) of those cars only which are above 500000 in price.

Practice Question 2

- Write a program that implements the following using C++ struct. The program should finally display the values stored in a phone directory (for 10 people)

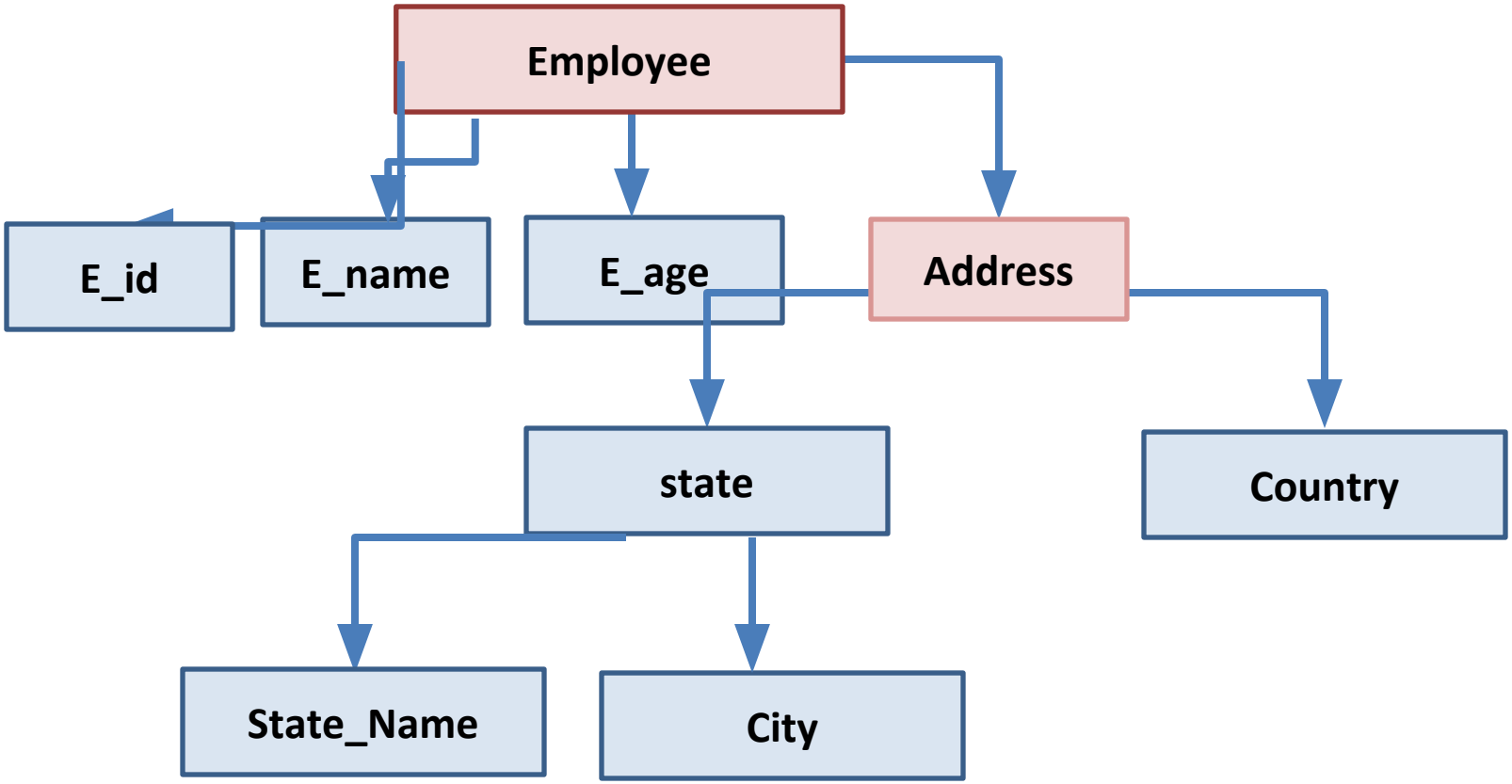


Write a C++ program that dynamically allocates an array of 10 Employees structures. Each Employee should have a emp_id,name, age, and an Address structure containing, country and state structure containing name ,city.

Take input for all 10 persons using pointers.

Print only the count of the employee whose city is "Islamabad".

At the end free the allocated memory in such a way there is no memory leakage problem.



References

- Chapter 11, Starting Out with C++ - From Control Structures through Objects-Pearson (2019)

Lecture slide credit goes to Dr. Akhtar Jamil from the National University of Computer and Emerging Sciences, Islamabad (Email: akhtar.jamil@nu.edu.pk).

Thank You ☺